

Jev Observer

A practical guide to starting the project

Free local software for understanding recurring Jev decisions.

Product recommendation: build a local proxy and log importer that recognizes recurring Jev questions, creates useful statistics, and makes definition changes visible.

Evidence base: a targeted static review of 100 public repositories, with an individual benefit hypothesis and integration constraint for every project.

Prepared: September 22, 2026

Current status: research and design. No application has been implemented, no compatibility or performance measurements have been completed, and no user interviews have been conducted.

Project: [LimePencil/jev-observer](#) — private during development, with an MIT license prepared for an eventual open-source release. Jev Observer is a working name and an independent project.

Local history, charts, labels and exports should be free. Live Jev inference still goes to the configured external provider and can incur its charges. Grouping should require no additional model calls.

01 / The decision to make

Build a Jev-specific question-statistics tool. Start with recurring validation or commit-check questions, then support one indexed-batch pattern. A broad proxy with a spend dashboard is less distinctive: several reviewed tools already have logs and dashboards.

The first promise should be: **“Your recurring Jev questions, with their own local statistics.”** A user points a compatible local client at Observer, or imports a supported log, and gets a meaningful view without configuring every chart.

Decisions already established

Decision	Working direction
Product	Jev-specific; Choice, Score and Noul awareness
Deployment	Local executable, browser UI, local database
Price and license	Free, MIT, no paid feature tiers
Data	No background analytics or automatic uploads
Repository	Private development now; public release is a separate future action
Initial collection	Native TypeSafe proxy plus local file import
First differentiation	Recurring question statistics and inspectable version changes

Reading map

Pages 3–5 explain the evidence, users and first experience. Pages 6–10 define grouping, statistics, architecture, privacy and costs. Pages 11–14 cover implementation, verification, launch and unresolved decisions. Page 15 links primary sources. Pages 16–40 contain all 100 project assessments, four per page.

What this report does not establish

Repository activity is not proof of production use, willingness to adopt another tool, or sustained demand. The reviewed sample includes related ports and evaluation projects. Proposed integrations, benefits and milestones still need hands-on validation. No outreach or publication has been performed.

02 / What Jev and the evidence tell us

Jev accepts input state and named questions through TypeSafe's system-one API. Its typed outputs create useful chart defaults: Choice selects among defined options; Score evaluates an ordered rubric; Noul reports a yes-probability. Instructions may themselves be structured. Responses associate answers with question keys and report usage for the request. [API reference](#).

Main grouping challenge in each sampled repository



The categories are exclusive manual primary assignments, not a complete inventory of every question. Fixed definitions and configurable rules account for 49 entries. Indexed instances and changing candidates account for 44. Seven primarily focus on controlled question or benchmark variants. These are sample counts, not ecosystem-wide percentages.

How the review was conducted

We selected 100 distinct repositories across eight categories from awesome-jev, inspected README sections and targeted implementation excerpts, and collected 531 source files at pinned commits. Supplemental request builders were examined when filename ranking missed the relevant implementation. Collected files are not a claim of exhaustive line-by-line review.

The sample has no GitHub-declared forks, but contains related ports and shared authors. Ten entries are evaluation-oriented, including a report/artifact study. The main nautilus-compass application is not counted as an established Jev runtime integration; its included work is a calibration study. No studied project or benchmark was executed. The appendix preserves these distinctions.

03 / Who should use it first

Prioritize developers who repeat the same semantic rule and change it over time. They offer a clearer first investigation than a complex autonomous browser loop.

Candidate	Useful first result	Main integration check
zod-jev — #9	Per-rule validation distributions and version changes	Test configurable base URL and issue mapping
jev-commit — #17	Five recurring checks, one parent cost record	Test transport and sensitive-source retention
jev-belay — #15	Check trends linked to existing labels	Import logs; complement its current live view
n8n node — #71	Charts per workflow and question	Verify self-hosted runtime can reach collector
jgrep — #7	One histogram across indexed rows	Add/test endpoint setting and family mapping

Numbers refer to the project appendix. These are candidates inferred from source review, not partners or confirmed users. Browser agents and compaction tools are valuable later fixtures because their identifiers and candidate sets change frequently.

Why a developer would keep it

They can answer a recurring question: “Did my rule change, did the inputs change, or did the model change?” The explorer should show the definition, alternatives, version and context needed to investigate. Measured correctness requires labels; comparing two dates alone does not prove regression.

Existing alternatives

Gateway logs, general observability platforms and application-specific dashboards already solve parts of this problem. LiteLLM documents TypeSafe pass-through; Vercel offers gateway observability; Langfuse offers tracing and evaluation. JevRouter, Scout and Belay already expose useful decisions. Sources are linked on page 15 and in appendix entries.

Compete on Jev-specific defaults and fewer setup steps. Offer imports and exports. Do not claim to be the first Jev proxy, to have exclusive access to probabilities, or to replace every existing dashboard. Validate whether Observer makes a concrete investigation easier.

04 / The first useful experience

A short user journey

1. Start Observer locally and open its bundled browser UI. Sample mode works without a provider key or internet access.
2. Connect a supported local client, or import a supported local JSONL file. Show the source name, collection status and any errors.
3. Send several inputs through the same named Jev rules. Observer creates recurring groups and suitable charts.
4. Open an unusual answer. Inspect its question definition, available distribution, parent request and cost basis.
5. Edit a rule. The next call appears under a separate definition version, with a visible comparison.
6. Attach a correction or import an application outcome. Keep this evidence distinct from the model's own answer.

Essential screens

View	What the user needs to see
Overview	Source, time window, requests, answer observations, errors, usage, cost basis, collector health
Question group	Definition, version, sample count, appropriate histogram or frequencies, model filter
Decision detail	Original question key, answer, alternatives, candidate context, linked action and label if supplied
Changes	Definition diff, presentation/task version, separate series and matched-example comparison when available
Data controls	Retention, payload capture, import/export, deletion, disk use

First release boundary

Include a native proxy, local history, automatic definition charts, file import/export, one indexed-family adapter and a synthetic demo. Add basic labels and richer comparisons next. Keep future improvements in the same free application.

Defer universal provider support, model routing, hosted accounts, collaboration servers, automatic prompt optimization and generic structured-output detection. These add scope without proving the core question-statistics workflow.

05 / How automatic grouping should work

Use the request definition, not just the answer shape. Two Noul outputs can represent entirely unrelated tasks. Recognition must be local, deterministic and inspectable.

Keep two levels of identity

Definition series: source namespace + original question key + complete definition fingerprint + presentation version + optional task-version discriminator. Preserve instructions, criteria, extra fields, strings and array order. A model version is a visible filter rather than a reason to discard history.

Question family: a reusable operation linking compatible instances or versions. Keep the original series underneath it. A family relationship is not permission to pool every statistic.

Important cases

Case	Required behavior
Same rule, new input	Accumulate answer observations under the recurring definition
Instructions or rubric changed	New definition version; show a diff
Presentation/order changed	Preserve a presentation version and separate default series
Renamed key	Suggest a relationship; keep original identity
Row/tool/pixel index changed	Use a tested source-specific adapter; retain instance reference
Candidate options changed	Preserve candidate-set fingerprint; scope label frequencies
Rules embedded in state	Use optional task-version metadata or selected configured state paths

Limits of “automatic”

If a question refers to rules in state, identical question text does not prove identical task meaning. Without a discriminator, label the group definition-based with unverified task-version context. Never hash all input state into identity: that makes every example a new task.

For unknown indexed patterns, offer a family suggestion and visible structural diff. Never remove all numbers or identifiers blindly: a changed age limit or threshold could be the rule itself. Version adapters and mapping reasons. Bound discovery work so a pixel workload cannot create thousands of visible charts.

06 / Statistics that mean what they say

Question	Default view	Interpretation boundary
Choice	Selected-option frequency and reported probability distributions	Only combine stable option meanings; candidate IDs may be request-local
Score	Histogram, percentiles and level distributions	Compare within the same rubric; a score is not an accuracy percentage
Noul	Yes-probability histogram and trend	Show threshold crossings only when the threshold is supplied
All	Answer count, distinct request count, associated latency and cost	Parent metrics overlap across groups; they are not exact per-question billing

Choice and Score confidence summarizes their probability distributions. Noul has no separate native confidence field. Preserve any application-derived metric with its origin; do not relabel it as a native field. [Confidence documentation](#).

Four records users must not confuse

1. **Model answer:** what Jev returned.
2. **Policy decision:** what the application selected after thresholds, overrides or combined scores.
3. **Execution result:** whether that action ran and what happened.
4. **Outcome label:** a human or process judgment of correctness or usefulness, with provenance.

A proxy directly observes the first. The other records require imports or application events. Cache hits, deterministic fast paths and heuristic decisions may never reach the proxy.

Denominators and comparisons

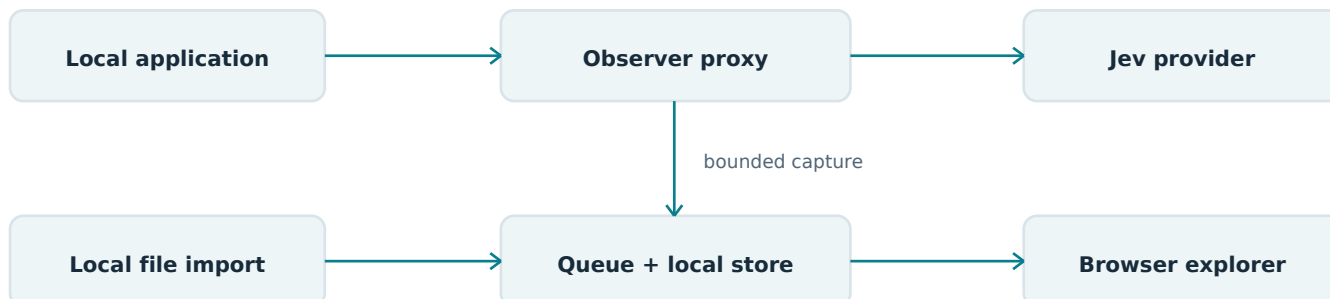
Twenty indexed questions in one request produce twenty answer observations and one parent request. A histogram weighted by answers differs from one weighted by requests; label the choice. Show missing and invalid answers separately from valid observations.

Keep timestamps, model/version, definition version, cohort and sample count visible. Distribution changes are descriptive unless compared on matched retained inputs. Labels from a teacher model are not independent ground truth. Reviewed samples can be biased; report label coverage and use a held-out representative set before recommending a policy threshold.

Unknown cost, missing labels, timeouts and absent execution outcomes must stay unknown.

07 / Proposed architecture

Use a small Rust service, SQLite and a bundled browser interface. This is a proposed implementation choice for a simple local package, not a benchmarked performance claim. Select and pin specific libraries when implementation starts.



Separate responsibilities

- **Forwarder:** accept the supported Jev route, forward to a configured upstream, preserve responses and transport outcomes.
- **Collector:** emit bounded request and answer events; expose queue drops, write failures and last successful capture.
- **Normalizer:** retain provider provenance, parse supported fields, preserve unknown fields or indicate when capture is incomplete.
- **Grouping engine:** compute definitions and families, apply versioned adapters and produce explainable mappings.
- **Store and importer:** validate events, migrate the local schema and preserve explicit source IDs.
- **Explorer:** query local aggregates and records without exposing provider credentials.

Failure behavior

Telemetry failure should be visible while forwarding continues according to documented behavior. A stopped proxy process still prevents traffic through it; asynchronous ingestion or file import is an alternative for applications that cannot accept that dependency.

Do not introduce an invisible extra retry layer. Record attempts separately and correlate them only with explicit identifiers. Preserve upstream errors; do not transform them into negative answers. Bound inspected/captured payload sizes and mark truncation. Exact limits need workload measurements.

Start with native TypeSafe HTTP. OpenRouter decisions, Vercel native evaluation and Cloudflare bindings need separate fixtures and normalization. Hosted runtimes cannot automatically reach a laptop's loopback listener.

08 / Data, privacy and local operations

Minimum local records

Record	Essential fields
Request	Local ID, source, timestamps, provider/model, status, usage, reported or estimated cost and rate version
Answer	Request ID, original key, type, definition/presentation IDs, value, available probabilities/confidence
Family mapping	Family and adapter versions, task/candidate discriminator, instance reference, reason
Action/outcome	Linked answer/request, policy version, attempted action, result or label, source and timestamp
Import	Import/source IDs, original event ID when available, original timestamp, format and provenance
Health	Dropped events, write failures, queue pressure, capture/redaction indicators

Compute account totals from the request ledger. Import deduplication requires explicit identity, not identical content: a second real call can be identical to the first. Keep database migrations versioned and export formats documented.

Retention and security defaults

- Bind to loopback. Use a configured upstream allowlist; avoid an arbitrary-URL forwarding endpoint.
- Keep provider keys out of the database, browser responses and exported records. Separate forwarding credentials from local UI access.
- Persist no raw input state by default. Make payload capture an explicit local choice with redaction and retention settings.
- Apply controls to question text, criteria and labels too. Hashes are useful identifiers, not anonymization guarantees.
- Bundle UI assets and disable automatic analytics/uploads. Defend local mutation endpoints against unwanted browser-origin requests.
- Provide local export, deletion and backups. Explain that deletion in the active database does not remove separate backups or user exports.

A record without retained input cannot support exact replay. Show that limitation. Re-evaluation is a new provider call; changing a threshold over already saved answers is an offline calculation. Review every outbound request in the release verification.

09 / What “free and local” costs

Observer should have no subscription, inference markup or feature gates. Local exploration, sample mode, export and offline calculations require no paid service. Users remain responsible for the external inference they choose to run.

Dated provider example

On September 22, 2026, TypeSafe lists Jev 1.13 at **\$0.042 per million input tokens**, with output tokens free. This is the direct provider's published rate, not a gateway-wide promise or invoice amount. Verify the applicable model/provider rate before a live run. [Model and pricing reference](#).

Requests	Mean input tokens	Illustrative inference spend
10,000	1,000	\$0.42
100,000	1,000	\$4.20
1,000,000	1,000	\$42.00
100,000	5,000	\$21.00

Formula: requests × mean input tokens ÷ 1,000,000 × rate. These examples exclude gateway-specific charges or discounts and assume usage is known. A multi-question request must not multiply its bill by the number of answers.

Local resource budget

Measure bytes per request and per answer, including candidate distributions and database indexes. An illustrative million bundles averaging 2 KB occupy about 2 GB before indexes and backups; 2 KB is an unmeasured assumption and may be far too small for large batches.

Use configurable retention, disk-usage reporting and bounded queues. Measure memory, latency and storage growth on fixed-question, indexed-batch and large-candidate fixtures. Publish hardware, sample size and workload with results. Do not advertise “no overhead” before measuring it.

Maintainer costs

The main commitment is engineering and maintenance: compatibility fixtures, packaging, migration tests and user support. A domain or hosted demo is optional. There is no proposed revenue target. Keep recurring infrastructure optional so users can build, inspect and run the software independently.

10 / Build in reviewable milestones

This is a sequence with exit criteria, not a delivery-date promise. Finish each thin slice before expanding provider or application coverage.

Milestone	Deliverable	Exit evidence
1. Fixtures and storage	Synthetic Choice/Score/Noul batches, event schema, local history and sample UI	Multiple answers reconcile to one request; offline browsing works
2. Native forwarding	TypeSafe pass-through, redaction and visible health	Success/errors preserved; secrets absent; pinned clients exercised
3. Recurring charts	Definition identity, basic distributions, presentation/version views	Stable rules accumulate; changed rules separate
4. Import and family	One existing log adapter and one indexed-batch adapter	Explicit IDs deduplicate; mapping is inspectable; request cost remains unique
5. Corrections and comparison	Labels, action links and version comparison	Unknowns visible; label provenance preserved; same-input comparison demonstrated
6. Release candidate	Build instructions, supported binaries, checksums and demo	Clean install, offline operation and documented limits verified

Suggested module boundaries

Keep forwarding, event storage, grouping and provider adapters separable. Put fixture data and adapter expectations beside the relevant modules. Bundle the UI only after its local data contract is stable. Prefer one executable and one local store over an initial service cluster.

First work session

Choose the first supported runtime and one integration fixture. Define the request/answer schema and a synthetic mixed batch. Implement storing and inspecting that batch offline before wiring live credentials. Then add native forwarding and exercise successful responses, errors and a storage failure.

Use the existing research repository and preserve its evidence. Pin dependencies and SDK versions in the implementation. Do not convert a source review into a compatibility claim until a real fixture has passed.

11 / What must be verified

Correctness and accounting

- One request with three questions creates one usage/cost record and three linked answer observations.
- Valid answers, missing answers, invalid bodies and provider failures have distinct states.
- Definition and presentation changes remain inspectable; shared shapes do not merge unrelated tasks.
- A recognized indexed batch creates one family while retaining all original definitions and instance references.
- Dynamic candidate IDs never become stable labels without a semantic mapping.
- Task-version metadata separates state-carried rule changes when supplied; absent context stays unverified.
- Reimporting the same explicit source event does not duplicate it; repeating a real call still records another observation.

Transport and operational behavior

Test success and error forwarding, timeouts, client retries, unknown fields, queue overflow, database write failure and process restart. Check that collection gaps are visible. Measure direct versus proxied behavior with representative payload sizes and concurrency before setting performance targets.

Test imports with malformed rows, unsupported formats, missing fields and duplicate IDs. Preserve a useful error report without exposing credentials or raw sensitive content. A failed row must not silently become a valid zero-valued observation.

Local trust and usability

Verify no credentials reach stored events, UI responses or exports. With networking disabled, sample mode, saved-history browsing and exports must still work. Inspect outbound connections for remote assets, analytics and account checks. Check retention and backup/export behavior.

Ask a developer to connect a supported workflow from a clean setup, find one unusual answer, understand its alternatives, and explain a changed rule. Record blockers through volunteered feedback. A proposed learning goal is four of five observed users reaching a useful decision within ten minutes; this is a target, not an achieved benchmark.

Release gate

Ship supported combinations with pinned versions, known limitations and reproducible examples. Mark everything else experimental or unverified. Passing local fixtures is necessary but does not establish accuracy, production reliability or broad SDK support.

12 / A launch that demonstrates usefulness

The message

Your recurring Jev questions, with their own local statistics. Show how quickly the tool answers a real question. “See what Jev chose. Catch what changed.” is a supporting message.

The first 90-second demonstration

1. Run a small synthetic validation or commit-check workflow.
2. Show named question groups appearing automatically across different inputs.
3. Inspect one answer and its parent request, usage and cost basis.
4. Edit a rule and compare its separate definition versions.
5. Show a labeled correction, then distinguish a saved-answer threshold preview from a new live evaluation.
6. Show an indexed batch: twenty answer observations, one recognizable family, one request cost.

Clearly label synthetic or recorded output. Publish live-measurement details only when live tests exist. Do not imply user endorsements or claim accuracy improvements from an attractive chart.

Distribution sequence

Prepare an MIT public release with local demo data, build instructions and supported binaries. Offer integration examples to relevant maintainers when outreach is separately authorized. Then consider Jev directories/community channels and a technical article about indexed grouping or request-cost accounting. A broad launch is more useful after developers can run the tool themselves.

No messages, submissions or promotions have been sent. The repository remains private. Its future public release is not accomplished by placing an MIT file in a private repository.

Evidence of adoption

Seek concrete debugging incidents, observed setup sessions and useful return sessions. Ask what the developer learned that their existing logs made difficult. Compare investigation steps, not just stated enthusiasm. Use voluntary feedback and public contribution activity; do not add background usage analytics.

If users only need basic statistics, keep the project small. If proxy setup repeatedly blocks adoption, prioritize import or asynchronous ingestion. If existing dashboards already answer the question, reconsider the proposed integration rather than manufacturing a gap.

13 / Risks and decisions still open

Risk	Response and next evidence
Grouping combines unrelated tasks	Strict definitions, versioned adapters, candidate context and optional task metadata
Similar instances fragment history	One indexed-family fixture; bounded suggestions for unknown formats
Existing dashboards are sufficient	Observe a real investigation; prioritize imports or narrower views
Local proxy cannot reach the workload	Verify runtime placement; offer local file import
Proxy adds failure or latency	Bound capture, expose health, measure overhead; document process dependency
Stored definitions leak sensitive content	Include questions, criteria and labels in retention/redaction controls
Cost or accuracy charts mislead	Preserve provenance, denominators, unknown values and label coverage
Upstream APIs or aliases change	Pin compatibility fixtures and record requested/returned model IDs
Early ecosystem attention fades	Seek repeat use before expanding scope

Resolve during implementation

Choose the first supported operating system and packaging target. Pick the Rust HTTP, database and UI libraries after checking their current releases and licensing. Set retention defaults and payload limits from measured fixtures. Select the first importer and its exact supported log version. Decide how users supply optional task-version metadata without modifying upstream API bodies.

The working name remains provisional. No naming clearance or TypeSafe affiliation has been established. Public release timing remains a separate decision.

Start here

1. Use zod-jev or jev-commit as the first compatibility candidate.
2. Build the offline mixed-batch explorer and correct request-level accounting.
3. Add native forwarding, then recurring charts.
4. Import one existing local log and normalize one indexed family.
5. Observe whether a developer can complete a useful investigation.

The next milestone is a running, verified local slice with the bounded scope described here.

14 / Sources and reproducibility

Official technical references

- [TypeSafe API](#): request/answer contract and usage. Rechecked September 22, 2026.
- [TypeSafe models](#): dated direct-provider pricing and model versions. Rechecked September 22, 2026.
- [TypeSafe confidence](#): native uncertainty fields and their interpretation. Rechecked September 22, 2026.
- [TypeSafe Python client](#): configurable client base URL.
- [Vercel TypeSafe API](#): one compatible gateway route; native evaluation is separate work.

Alternatives and discovery

- [LiteLLM TypeSafe pass-through](#), [Vercel observability](#), [Langfuse observability](#): documented substitutes, not hands-on comparative results.
- [awesome-jev discovery snapshot](#): source directory for the purposive sample.

Research package in the repository

The research/survey folder contains the readable findings, 100-row matrix, CSV, manual annotations, sample list, source manifest, discovery record and reproduction scripts. Source links in the appendix are pinned to the reviewed commits. Full third-party source snapshots remain outside this repository.

The build_report.py script validates 100 unique annotations and their evidence paths, then generates the matrix and counts. replay_sources.py retrieves the exact manifest files and verifies decoded-text hashes without executing studied projects. All 531 cached source hashes were checked; three fresh downloads were also verified.

The PDF is generated from this guide and the same survey annotations. Rebuild instructions and dependency versions are in reports/README.md. The accompanying Markdown research is editable; this PDF is a dated snapshot. Primary-source links may require internet access, and the private project repository requires permission to view.

How to read the appendix

Each card separates observed use, existing visibility, proposed help and integration limits. “Observed” means static source/documentation evidence, not successful execution. Principal implementation and README links appear on every card; additional collected files are listed in the manifest. One primary grouping pattern is assigned per repository, even when it uses several.

Appendix / Projects 001-004

Static review · Proposed benefits are hypotheses · Code and README links are pinned to reviewed commits.

001 carldaws/hunch

SDKs and developer tools / Configured rules

Use: Ruby predicates, choices and scores with configurable question text and thresholds.

Already visible: Returned probabilities and threshold helpers.

Observer could help: Group recurring predicates and show how frequently each crosses its configured threshold.

Limit: Application events are needed to know which branch actually ran.

[Implementation evidence](#) · [Project README](#)

002 can1357/jegrep

SDKs and developer tools / Changing candidates

Use: Semantic source search over changing files and search candidates.

Already visible: JSON reports and benchmark records.

Observer could help: Compare search-mode distributions, abstentions and batch costs across runs.

Limit: Search queries and candidate sets change; support both configured providers explicitly.

[Implementation evidence](#) · [Project README](#)

003 formulahendry/jev-acp

SDKs and developer tools / Configured rules

Use: ACP agent constructs typed questions from tools and user-supplied definitions.

Already visible: Session history command and decision responses.

Observer could help: Persist per-template statistics across editor sessions.

Limit: Ad hoc questions need their own definitions rather than one shared ACP group.

[Implementation evidence](#) · [Project README](#)

004 blakestone-x/jev-mcp

SDKs and developer tools / Configured rules

Use: MCP tools classify, score, check, match and screen using typed questions.

Already visible: Tool responses expose structured answers.

Observer could help: Separate histories by tool and rule, with threshold and score distributions.

Limit: Persistence beyond responses was not established in the reviewed slice.

[Implementation evidence](#) · [Project README](#)

Appendix / Projects 005-008

Static review · Proposed benefits are hypotheses · Code and README links are pinned to reviewed commits.

005 mrnugget/jev-shell-history

SDKs and developer tools / Changing candidates

Use: Ranks changing shell-history candidates with a completion gate.

Already visible: Inline suggestions and CLI usage output.

Observer could help: Show abstention, uncertainty and latency by suggestion mode.

Limit: Candidate IDs only identify commands within that request; selected suggestions need application events.

[Implementation evidence](#) · [Project README](#)

006 allebee/jevgrep

SDKs and developer tools / Indexed/contextual instances

Use: Batches per-line semantic predicates with line references in instructions.

Already visible: CLI JSON and benchmark cost records.

Observer could help: Combine indexed line questions into per-condition distributions.

Limit: User conditions change and the default gateway needs a tested adapter.

[Implementation evidence](#) · [Project README](#)

007 kyu1204/jgrep

SDKs and developer tools / Indexed/contextual instances

Use: Filters code chunks and structured rows with indexed question keys and references.

Already visible: CLI results and cache files.

Observer could help: Recognize repeated row or chunk rules independently of batch position.

Limit: Endpoint configuration and cache-event import require work.

[Implementation evidence](#) · [Project README](#)

008 allebee/pytest-jev

SDKs and developer tools / Configured rules

Use: Semantic test assertions evaluate configured claims against thresholds.

Already visible: Pytest assertion reports and benchmark artifacts.

Observer could help: Track per-assertion probability changes between test runs.

Limit: Positive and negative assertions have different threshold semantics.

[Implementation evidence](#) · [Project README](#)

Appendix / Projects 009-012

Static review · Proposed benefits are hypotheses · Code and README links are pinned to reviewed commits.

009 jomatsu/zod-jev

SDKs and developer tools / Configured rules

Use: Batches semantic Zod rules and distinguishes rejection from uncertainty.

Already visible: Typed rule issues and decision results.

Observer could help: Automatically build per-rule distributions and compare rule versions.

Limit: Base URL is configurable; linking parse outcomes still requires events.

[Implementation evidence](#) · [Project README](#)

010 ellipsis-dev/blink

Agents, coding, and guardrails / Changing candidates

Use: Filesystem search walkers repeatedly choose among changing directory entries.

Already visible: Walker traces and estimated cost.

Observer could help: Compare branch selection and uncertainty across search sessions.

Limit: Keep walker identity and request-local path mappings.

[Implementation evidence](#) · [Project README](#)

011 qkal/Canny

Agents, coding, and guardrails / Fixed definitions

Use: Optional Jev advice augments deterministic completion checks.

Already visible: Evidence ledger and replay workflow.

Observer could help: Track advisory probabilities alongside recorded completion outcomes.

Limit: Do not describe advisory answers as the deterministic blocking policy; endpoint is configurable.

[Implementation evidence](#) · [Project README](#)

012 devanshbatham/commit-miner

Agents, coding, and guardrails / Configured rules

Use: Classifies commits using configurable change, security and vulnerability taxonomies.

Already visible: HTML and CSV reports.

Observer could help: Show recurring taxonomy distributions by repository and definition version.

Limit: Taxonomy edits change meaning; reviewer corrections are separate labels.

[Implementation evidence](#) · [Project README](#)

Appendix / Projects 013-016

Static review · Proposed benefits are hypotheses · Code and README links are pinned to reviewed commits.

013 thruwire/foreman

Agents, coding, and guardrails / Fixed definitions

Use: Assesses worker progress, completeness, tests and need for human intervention.

Already visible: Policy decisions and CLI workflow.

Observer could help: Show assessment trajectories per job and connect them to worker outcomes.

Limit: Jev assessments and deterministic policy actions must remain distinct.

[Implementation evidence](#) · [Project README](#)

014 luantak/is-malicious

Agents, coding, and guardrails / Fixed definitions

Use: Applies named security predicates to source chunks.

Already visible: Reports include decisions and usage or estimated cost.

Observer could help: Track recurring check distributions and reviewed false positives.

Limit: Its derived Noul confidence must not be mistaken for an upstream confidence field.

[Implementation evidence](#) · [Project README](#)

015 valentyntkit/jev-belay

Agents, coding, and guardrails / Fixed definitions

Use: A stop hook batches fixed questions before applying completion policy.

Already visible: JSONL decisions, shadow mode, labels and live inspection.

Observer could help: Compare question and policy versions across sessions using existing labels.

Limit: Already has useful visibility; integrate logs and configurable base URL rather than replace the UI.

[Implementation evidence](#) · [Project README](#)

016 0xNatoshi/jev-codex-router

Agents, coding, and guardrails / Fixed definitions

Use: Selects coding models and effort with deterministic routing policy.

Already visible: Local decision JSONL and probability distributions.

Observer could help: Compare raw question answers with the model route actually chosen.

Limit: Savings and task quality need measured outcomes, not routing probabilities.

[Implementation evidence](#) · [Project README](#)

Appendix / Projects 017-020

Static review · Proposed benefits are hypotheses · Code and README links are pinned to reviewed commits.

017 valentynkit/jev-commit

Agents, coding, and guardrails / Fixed definitions

Use: Batches five commit checks over staged changes.

Already visible: CLI verdicts and cost reporting.

Observer could help: Create recurring check charts and show which definition changed rejection rates.

Limit: Configurable base URL makes this a candidate fixture; staged source should not be retained by default.

[Implementation evidence](#) · [Project README](#)

018 eugeniughelbur/jev-engineering

Agents, coding, and guardrails / Fixed definitions

Use: Combines destructive-action checks and verdict choices with deterministic gates.

Already visible: Decision JSONL and calibration tools.

Observer could help: Compare check distributions and threshold policies over time.

Limit: Fast paths never reaching Jev require application events; gateway protocol needs testing.

[Implementation evidence](#) · [Project README](#)

019 leepokai/jev-guard

Agents, coding, and guardrails / Fixed definitions

Use: Evaluates tool risk, user intent and untrusted content before guard decisions.

Already visible: Extension verdicts; documentation emphasizes avoiding retained author data.

Observer could help: Offer opt-in local histories for individual guard checks.

Limit: Respect existing retention expectations and support provider dialects explicitly.

[Implementation evidence](#) · [Project README](#)

020 doeixd/jev-pref

Agents, coding, and guardrails / Configured rules

Use: Turns user preference rules into named semantic classifications.

Already visible: Evaluation and tuning workflows.

Observer could help: Track rule-version distributions and link existing evaluation labels.

Limit: Configurable base URL helps integration; preference text can be sensitive.

[Implementation evidence](#) · [Project README](#)

Appendix / Projects 021-024

Static review · Proposed benefits are hypotheses · Code and README links are pinned to reviewed commits.

021 devagrawal09/jev-review

Agents, coding, and guardrails / Changing candidates

Use: Reviews staged patches with broad checks and focused region judgments.

Already visible: Local review dashboard and stored reports.

Observer could help: Add cross-review question trends and definition comparisons.

Limit: Dynamic hunk or region identifiers need mappings; existing dashboard reduces incremental value.

[Implementation evidence](#) · [Project README](#)

022 NiazMorshed2007/jev-review

Agents, coding, and guardrails / Fixed definitions

Use: MCP quality review combines applicability, scalar dimensions and classifications.

Already visible: Structured MCP responses.

Observer could help: Track quality dimensions over time with applicability-aware denominators.

Limit: A non-applicable dimension is not a low score; endpoint is fixed in reviewed client.

[Implementation evidence](#) · [Project README](#)

023 gargpratyush/jev-router

Agents, coding, and guardrails / Changing candidates

Use: Routes turns among an eligible model roster, with confidence-dependent policy.

Already visible: Router CLI status and local operational records.

Observer could help: Compare model-choice distributions with fallback or override decisions.

Limit: Changing eligible models alter the choice domain.

[Implementation evidence](#) · [Project README](#)

024 BillionsBobby/JevRouter

Agents, coding, and guardrails / Changing candidates

Use: Selects capabilities across tools, models and skills in execution plans.

Already visible: Decision receipts and dashboard.

Observer could help: Compare question versions and recurring capability families across plans.

Limit: Prefer receipt interoperability; much of the basic explorer already exists.

[Implementation evidence](#) · [Project README](#)

Appendix / Projects 025-028

Static review · Proposed benefits are hypotheses · Code and README links are pinned to reviewed commits.

025 kierandotai/jev-scout

Agents, coding, and guardrails / Fixed definitions

Use: Scores search and fetch results with versioned relevance and credibility rubrics.

Already visible: JSONL, live dashboard and cached-score gates.

Observer could help: Compare rubric versions and distributions across retrieval sessions.

Limit: Thresholds applied to cached scores are invisible to an HTTP-only proxy.

[Implementation evidence](#) · [Project README](#)

026 shimo4228/jev-skill-router

Agents, coding, and guardrails / Changing candidates

Use: Selects among installed skills and evaluates supporting gates.

Already visible: Prompt-hash JSONL and shadow or injection modes.

Observer could help: Compare roster versions and proposed versus actually used skills.

Limit: Keep dynamic roster semantics and respect base-URL security checks.

[Implementation evidence](#) · [Project README](#)

027 xinyao27/jevonian

Agents, coding, and guardrails / Indexed/contextual instances

Use: Combines model routing with per-tool context compaction judgments.

Already visible: Local ledger and dashboard.

Observer could help: Add question-family drilldowns and compaction template comparisons.

Limit: Routing and compaction are separate families; several provider dialects are present.

[Implementation evidence](#) · [Project README](#)

028 lakeday-org/perch

Agents, coding, and guardrails / Configured rules

Use: Semantic linter uses configured rules and changing source-location choices.

Already visible: Closed-finding JSONL with dismissal reasons.

Observer could help: Link dismissed findings to per-rule histories and version changes.

Limit: Location choices need request-local mappings; existing labels can be imported.

[Implementation evidence](#) · [Project README](#)

Appendix / Projects 029-032

Static review · Proposed benefits are hypotheses · Code and README links are pinned to reviewed commits.

029 y0usaf/pi-jev

Agents, coding, and guardrails / Fixed definitions

Use: Pi extension evaluates tool-action gates and output failure categories.

Already visible: In-session verdict UI and structured results.

Observer could help: Separate recurring guard checks from arbitrary user questions.

Limit: Actual allow or hold behavior depends on application policy.

[Implementation evidence](#) · [Project README](#)

030 DevMortimer/pi-warden

Agents, coding, and guardrails / Fixed definitions

Use: Pi agent guard performs action, completion, rule and output checks.

Already visible: SQLite traces, sidebar and hold statistics.

Observer could help: Compare checks and versions using local trace import.

Limit: Backend delegates through pi-typesafe; direct proxy configuration is not established.

[Implementation evidence](#) · [Project README](#)

031 Dicklesworthstone/skillranker

Agents, coding, and guardrails / Changing candidates

Use: Ranks skill candidates and tests fit with skill content embedded in instructions.

Already visible: Optional ledger and why-not explanations.

Observer could help: Group fit templates and compare candidate rosters across searches.

Limit: A tested adapter must distinguish contextual skill content from the reusable rule.

[Implementation evidence](#) · [Project README](#)

032 lukstei/slop-grader

Agents, coding, and guardrails / Configured rules

Use: Semantic grading uses configurable rules and caches their definitions.

Already visible: CLI statistics and grading reports.

Observer could help: Show per-rule distributions and changes across document revisions.

Limit: Cached results need import; multiple provider formats need compatibility checks.

[Implementation evidence](#) · [Project README](#)

Appendix / Projects 033-036

Static review · Proposed benefits are hypotheses · Code and README links are pinned to reviewed commits.

033 supercorp-ai/supercov

Agents, coding, and guardrails / Fixed definitions

Use: Assesses source files using a fixed collection of named quality predicates.

Already visible: Quality property files alongside coverage records.

Observer could help: Compare property probabilities across source revisions.

Limit: Jev quality scores are separate from measured test coverage.

[Implementation evidence](#) · [Project README](#)

034 mblode/taste-lint

Agents, coding, and guardrails / Configured rules

Use: Builds semantic taste checks from configurable rule definitions.

Already visible: Profile scorecards.

Observer could help: Compare rule and profile versions and attach reviewed false positives.

Limit: TypeSafe and native Vercel evaluation use different request formats.

[Implementation evidence](#) · [Project README](#)

035 shitianfang/wakegate

Agents, coding, and guardrails / Fixed definitions

Use: Produces bounded wake signals consumed by threshold and fallback policy.

Already visible: Evaluation scenarios and wake decisions.

Observer could help: Track signal distributions versus actual wake events and corrections.

Limit: Remote deployment needs export or local execution; error fallbacks are not model answers.

[Implementation evidence](#) · [Project README](#)

036 leonaaardob/fast-dev-compaction

Context and compaction / Indexed/contextual instances

Use: Codex compaction port asks about retaining tool calls and their results.

Already visible: Compaction statistics and recovery workflow described by the project.

Observer could help: Group tool/result predicates despite changing call IDs and lengths.

Limit: Related port, not independent demand; README labels the work a proof of concept.

[Implementation evidence](#) · [Project README](#)

Appendix / Projects 037-040

Static review · Proposed benefits are hypotheses · Code and README links are pinned to reviewed commits.

037 tamaratran/fast-jev-compaction

Context and compaction / Indexed/contextual instances

Use: Claude compaction asks two retention predicates for each tool call.

Already visible: Compaction statistics and recovery controls.

Observer could help: Compare retention distributions by tool and connect restore events.

Limit: Instructions include call metadata; configurable base URL still needs integration testing.

[Implementation evidence](#) · [Project README](#)

038 joelhooks/pi-fast-jev-compaction

Context and compaction / Indexed/contextual instances

Use: Pi port uses per-tool call/result retention questions.

Already visible: Ledger, session branching, status and cached usage.

Observer could help: Compare retention families across sessions and branches.

Limit: Related implementation lineage; actual saved tokens require application measurements.

[Implementation evidence](#) · [Project README](#)

039 ilkerulusoy/pi-jev-compact

Context and compaction / Indexed/contextual instances

Use: Pi compaction evaluates tool and prose retention.

Already visible: Traces and reports with distributions.

Observer could help: Compare templates and connect recovery or restoration outcomes.

Limit: Existing reporting overlaps; end-user endpoint override is not established.

[Implementation evidence](#) · [Project README](#)

040 kevinpita/pi-jev-context

Context and compaction / Fixed definitions

Use: Reversibly filters older context according to retention judgments.

Already visible: Session history remains available; context state is cached.

Observer could help: Track probability changes by retention mode and definition version.

Limit: An application event is needed to know what was actually hidden.

[Implementation evidence](#) · [Project README](#)

Appendix / Projects 041-044

Static review · Proposed benefits are hypotheses · Code and README links are pinned to reviewed commits.

041 GhalebDweikat/winnow

Context and compaction / Indexed/contextual instances

Use: Evaluates output blocks for relevance and preservation conditions.

Already visible: Cache and restore workflow.

Observer could help: Track retained versus dropped families and later restorations.

Limit: Block references require an adapter; separate summarizer usage must not be attributed to Jev.

[Implementation evidence](#) · [Project README](#)

042 compozy/yoshi

Context and compaction / Indexed/contextual instances

Use: Pruning proxy judges candidate relevance, coverage and contradiction.

Already visible: Benchmark, calibration and trace artifacts.

Observer could help: Make recurring pruning questions comparable across workloads and versions.

Limit: Existing measurements are a baseline; no savings benefit was independently verified.

[Implementation evidence](#) · [Project README](#)

043 GoldenLoaf24h/browserclaw

Browser and computer use / Changing candidates

Use: Browser controller chooses among observed targets and action gates.

Already visible: Native execution loop with a separate heuristic engine.

Observer could help: Track operation families and distinguish heuristic from model decisions.

Limit: Target IDs are local to a page snapshot; effects require application events.

[Implementation evidence](#) · [Project README](#)

044 jkudish/jev-browser

Browser and computer use / Changing candidates

Use: Browser loop selects available actions and checks goal or stuck state.

Already visible: Step traces and cost reporting.

Observer could help: Show repeated loop failures, uncertainty and candidate-aware action histories.

Limit: Element identifiers cannot become global option labels.

[Implementation evidence](#) · [Project README](#)

Appendix / Projects 045-048

Static review · Proposed benefits are hypotheses · Code and README links are pinned to reviewed commits.

045 wy-coliney/jev-browser-use

Browser and computer use / Changing candidates

Use: Browser skill selects allowed actions including done or blocked alternatives.

Already visible: Agent-visible decisions and a verification workflow.

Observer could help: Compare action uncertainty with subsequent verification failures.

Limit: Provider adapters and explicit verification-event links are needed.

[Implementation evidence](#) · [Project README](#)

046 socai-io/jev-social

Browser and computer use / Changing candidates

Use: Classifies a social platform then selects actions over observed targets.

Already visible: Action history in the browser workflow.

Observer could help: Keep platform classification and action-selection families distinct.

Limit: Target sets and action outcomes change across pages.

[Implementation evidence](#) · [Project README](#)

047 browser-use/jev-ultrafast

Browser and computer use / Changing candidates

Use: Browser agent chooses operations and elements using goal-dependent instructions.

Already visible: Example traces and interactive execution.

Observer could help: Recognize operation families while preserving goal and candidate context.

Limit: Goal text embedded in instructions cannot be blindly removed by generic normalization.

[Implementation evidence](#) · [Project README](#)

048 moritzkremb/jev-voice-browser

Browser and computer use / Changing candidates

Use: Voice browser evaluates partial speech, intent, site and target questions.

Already visible: Live decision display and policy decisions.

Observer could help: Compare partial versus final utterance decisions and cancellation rates.

Limit: Confirmation and actual execution require application events.

[Implementation evidence](#) · [Project README](#)

Appendix / Projects 049-052

Static review · Proposed benefits are hypotheses · Code and README links are pinned to reviewed commits.

049 chy4pro/jev-for-chrome

Browser and computer use / Changing candidates

Use: Chrome browser-control port requests typed actions and page targets.

Already visible: Popup trace inspection and export.

Observer could help: Import runs for cross-session and definition-version comparisons.

Limit: Related browser-agent lineage; extension permissions and provider settings need testing.

[Implementation evidence](#) · [Project README](#)

050 droidrun/mobile-jev

Browser and computer use / Changing candidates

Use: Android controller selects bounded operations and screen targets.

Already visible: Studio UI and JSONL traces.

Observer could help: Compare operation uncertainty and failure loops across device sessions.

Limit: Keep screen-local element mappings and import execution results.

[Implementation evidence](#) · [Project README](#)

051 awlevin/typesafe-computer-use

Browser and computer use / Changing candidates

Use: Mac computer-use loop selects bounded actions and OCR targets.

Already visible: Logs, reports and annotated screenshots described in the project.

Observer could help: Compare action phases and link uncertain targets with execution failures.

Limit: OCR candidates are request-local; screenshot retention must remain opt-in.

[Implementation evidence](#) · [Project README](#)

052 colliber/duckdb-jev

Routing, data, and workflows / Configured rules

Use: DuckDB extension builds typed questions for semantic SQL operations.

Already visible: SQL results and usage/cache counters.

Observer could help: Group repeated SQL predicates and track remote request costs across queries.

Limit: A proxy cannot observe reuse inside the database cache.

[Implementation evidence](#) · [Project README](#)

Appendix / Projects 053-056

Static review · Proposed benefits are hypotheses · Code and README links are pinned to reviewed commits.

053 AboveColin/HA-Jev

Routing, data, and workflows / Configured rules

Use: Home Assistant configures typed sensor judgments and assist behavior.

Already visible: Persisted usage and native sensor histories.

Observer could help: Track definition changes and decision distributions across sensors.

Limit: Home Assistant already provides time-series views; added value needs validation.

[Implementation evidence](#) · [Project README](#)

054 yusukebe/hono-jev-router

Routing, data, and workflows / Indexed/contextual instances

Use: HTTP router batches per-route predicates then applies first-match policy.

Already visible: Route results and playground examples.

Observer could help: Show ambiguous multi-match cases and distributions per route definition.

Limit: The chosen route and threshold are application policy; base URL is configurable.

[Implementation evidence](#) · [Project README](#)

055 sutro-sh/jev-align

Routing, data, and workflows / Configured rules

Use: Active-learning workflow edits Jev definitions using human labels.

Already visible: Optimization runs and registry workflow.

Observer could help: Compare deployed rule versions with imported labeling data.

Limit: Complement its optimizer; do not introduce another optimization engine.

[Implementation evidence](#) · [Project README](#)

056 AkashPriyadarshii/jev-curate

Routing, data, and workflows / Configured rules

Use: Filters datasets using predicate and score presets with confidence floors.

Already visible: Accepted and rejected JSONL outputs.

Observer could help: Compare rejection reasons and distributions across preset versions.

Limit: Accepted output is not evidence of correctness without labels.

[Implementation evidence](#) · [Project README](#)

Appendix / Projects 057-060

Static review · Proposed benefits are hypotheses · Code and README links are pinned to reviewed commits.

057 jyatesdotdev/jev-logtrriage

Routing, data, and workflows / Fixed definitions

Use: Batches six log judgments before choosing an alert or suppression action.

Already visible: SQLite cache and reports.

Observer could help: Build recurring check charts and compare alert-policy versions.

Limit: Cached decisions and actual alert actions need import or application events.

[Implementation evidence](#) · [Project README](#)

058 fazlerocks/jevmail

Routing, data, and workflows / Fixed definitions

Use: Classifies email category, urgency and need for human involvement.

Already visible: SQLite classification records and feedback schema.

Observer could help: Compare question versions and uncertainty against existing corrections.

Limit: Native Vercel evaluation needs an adapter; preserve email-retention choices.

[Implementation evidence](#) · [Project README](#)

059 hotchpotch/jev-reranker

Routing, data, and workflows / Indexed/contextual instances

Use: Evaluates document relevance or evidence, including pairwise variants.

Already visible: Probability-bearing results and evaluation examples.

Observer could help: Group document predicates by mode and rubric across queries.

Limit: Candidate references need normalization scoped to a tested adapter.

[Implementation evidence](#) · [Project README](#)

060 shinpr/jev-reranker

Routing, data, and workflows / Indexed/contextual instances

Use: Rust reranker filters, ranks and compresses candidate text using predicates.

Already visible: CLI JSON ranking results.

Observer could help: Compare recurring decision operations and threshold distributions.

Limit: Document references vary and ranking scores need labels before accuracy claims.

[Implementation evidence](#) · [Project README](#)

Appendix / Projects 061-064

Static review · Proposed benefits are hypotheses · Code and README links are pinned to reviewed commits.

061 choxos/jev-reviewer

Routing, data, and workflows / Configured rules

Use: Document review uses configurable questions to select and verify passages.

Already visible: Quoted evidence, citations and review interface.

Observer could help: Track each review template across documents and attach reviewer labels.

Limit: Browser relay configuration needs testing; sensitive source documents should remain unretained by default.

[Implementation evidence](#) · [Project README](#)

062 superagents-lab/jev-search

Routing, data, and workflows / Changing candidates

Use: Search app chooses sources and time windows, then judges results.

Already visible: Scored search interface and provider fallback logic.

Observer could help: Separate search-routing from result-quality histories and provider attempts.

Limit: TypeSafe, Vercel and Cloudflare bindings need distinct integrations; remote runtime cannot use desktop loopback.

[Implementation evidence](#) · [Project README](#)

063 Cab14bacc/jev-sheets

Routing, data, and workflows / Configured rules

Use: Spreadsheet functions create user-defined Noul, Choice and Score questions.

Already visible: Cell outputs and examples.

Observer could help: Import decision records for per-formula or rule histories.

Limit: Hosted Apps Script cannot reach a user's loopback proxy directly.

[Implementation evidence](#) · [Project README](#)

064 reachjalil/jev-tree

Routing, data, and workflows / Changing candidates

Use: Traverses a hierarchy of bounded choices to select a leaf.

Already visible: CLI and synthetic benchmark output.

Observer could help: Compare uncertainty, path depth and branch decisions.

Limit: Option meaning is node-specific; preserve tree version and path.

[Implementation evidence](#) · [Project README](#)

Appendix / Projects 065-068

Static review · Proposed benefits are hypotheses · Code and README links are pinned to reviewed commits.

065 replynodes/jev-web-analyzer

Routing, data, and workflows / Fixed definitions

Use: Website analyzer batches a fixed checklist with optional custom questions.

Already visible: Streamed scorecards, timings and usage.

Observer could help: Compare sites and checklist versions across runs.

Limit: Hosted route needs export or local self-deployment; custom questions remain separate.

[Implementation evidence](#) · [Project README](#)

066 gaborishka/jev-wrapped

Routing, data, and workflows / Fixed definitions

Use: Classifies Telegram posts and evaluates advertising, clickbait and pressure.

Already visible: Aggregated cards, trends and cloud persistence.

Observer could help: Add question-version comparisons and request-level error or cost history.

Limit: Already has domain statistics; remote deployment requires import or a local backend.

[Implementation evidence](#) · [Project README](#)

067 reachjalil/jevlogs

Routing, data, and workflows / Fixed definitions

Use: Enriches logs with incident-related boolean, urgency and information judgments.

Already visible: OpenTelemetry-linked records and benchmark reports.

Observer could help: Connect recurring question statistics with existing trace IDs.

Limit: Native gateway boolean format requires normalization; preserve field provenance.

[Implementation evidence](#) · [Project README](#)

068 kylemclaren/jevql

Routing, data, and workflows / Configured rules

Use: Runs typed semantic SQL row judgments through a Postgres-oriented client.

Already visible: Query results and usage totals.

Observer could help: Group recurring predicate definitions and compare query runs.

Limit: Distinguish application-derived Noul confidence from native fields and account for batching.

[Implementation evidence](#) · [Project README](#)

Appendix / Projects 069-072

Static review · Proposed benefits are hypotheses · Code and README links are pinned to reviewed commits.

069 EugeneBoondock/jevsq1

Routing, data, and workflows / Configured rules

Use: Semantic SQL service manages versioned decision policies and evaluations.

Already visible: Labels, evaluation and telemetry are already part of its service.

Observer could help: Offer portable local inspection and shared event exports.

Limit: Substantial overlap; this is a lower-priority early integration.

[Implementation evidence](#) · [Project README](#)

070 WiktorB2004/llama-index-jev

Routing, data, and workflows / Changing candidates

Use: Framework integration selects tools and reranks with typed judgments.

Already visible: Selector results, scores and confidence fallback.

Observer could help: Compare stages and abstentions while retaining candidate descriptions.

Limit: OpenRouter transport and framework result mappings need tests.

[Implementation evidence](#) · [Project README](#)

071 n3ndor/n8n-nodes-typesafe-jev

Routing, data, and workflows / Configured rules

Use: Workflow node builds configured multi-question Jev requests.

Already visible: Node execution outputs and request metadata.

Observer could help: Create recurring charts per workflow and definition without per-call tags.

Limit: Credential base URL is configurable; self-hosted workflow runtime must be able to reach the collector.

[Implementation evidence](#) · [Project README](#)

072 realZachi/pg-jev

Routing, data, and workflows / Indexed/contextual instances

Use: Postgres functions batch semantic row decisions with record references.

Already visible: Session cache and usage statistics.

Observer could help: Group record questions by SQL predicate rather than row index.

Limit: Configurable API URL helps; database cache activity needs a separate import.

[Implementation evidence](#) · [Project README](#)

Appendix / Projects 073-076

Static review · Proposed benefits are hypotheses · Code and README links are pinned to reviewed commits.

073 kyotofin/tax-doc-classifier

Routing, data, and workflows / Fixed definitions

Use: Classifies document pages using form and page-kind taxonomies.

Already visible: Corpus evaluation and confidence-gate reports.

Observer could help: Compare taxonomy versions and confidence against reviewed page labels.

Limit: Sensitive documents need explicit retention; published benchmark claims were not reproduced.

[Implementation evidence](#) · [Project README](#)

074 iamvatsalpatel/tiershift

Routing, data, and workflows / Fixed definitions

Use: Routes models from repeated difficulty, stakes, reasoning and domain signals.

Already visible: Decision JSONL and CLI analytics or cost comparisons.

Observer could help: Drill into individual signal versions and downstream route overrides.

Limit: Existing routing analytics overlap; task quality requires outcome records.

[Implementation evidence](#) · [Project README](#)

075 AbdelStark/heist-one

Games, robotics, and interactive demos / Fixed definitions

Use: Game guards judge suspicion, intent and attention.

Already visible: DecisionLens JSONL and game telemetry.

Observer could help: Compare behavior distributions across game sessions and rule versions.

Limit: Simulated character judgments do not establish real-world behavioral accuracy.

[Implementation evidence](#) · [Project README](#)

076 gaborishka/jev-canvas

Games, robotics, and interactive demos / Changing candidates

Use: Voice canvas chooses actions, shapes and colors with dynamic canvas targets.

Already visible: Interactive canvas feedback.

Observer could help: Track partial-speech decisions and link corrections to executed edits.

Limit: Target shapes change; preserve their request-local meaning.

[Implementation evidence](#) · [Project README](#)

Appendix / Projects 077-080

Static review · Proposed benefits are hypotheses · Code and README links are pinned to reviewed commits.

077 RomanSlack/jev-drone

Games, robotics, and interactive demos / Fixed definitions

Use: Simulated drone selects bounded tactics with risk and supporting judgments.

Already visible: HUD, telemetry and replay traces.

Observer could help: Compare tactical decisions by flight phase and rule version.

Limit: Jev is not the entire flight-control loop; physical-flight benefit was not tested.

[Implementation evidence](#) · [Project README](#)

078 dabit3/jev-experiments

Games, robotics, and interactive demos / Changing candidates

Use: Demo collection includes support-assist macro choices and other typed workflows.

Already visible: Per-demo user interfaces and decision responses.

Observer could help: Use separate source namespaces and compare recurring questions within each app.

Limit: Counted as one repository, not several independent adoption signals.

[Implementation evidence](#) · [Project README](#)

079 amycardoso/jev-palette

Games, robotics, and interactive demos / Fixed definitions

Use: Ranks a fixed command palette using a complete choice distribution.

Already visible: Interactive ranking and confidence display.

Observer could help: Compare probability mass and ranking stability with command clicks.

Limit: Argmax frequency alone loses its ranking use case; clicks need application events.

[Implementation evidence](#) · [Project README](#)

080 valentynkit/jev-plays-pokemon-red

Games, robotics, and interactive demos / Changing candidates

Use: Game agent chooses legal moves and predicts faint or flee conditions.

Already visible: Local JSONL, overlays and calibration outcomes.

Observer could help: Compare phases and predicted outcomes with actual game events.

Limit: Dynamic legal moves require candidate snapshots; base URL is configurable.

[Implementation evidence](#) · [Project README](#)

Appendix / Projects 081-084

Static review · Proposed benefits are hypotheses · Code and README links are pinned to reviewed commits.

081 thelau/jev-tetris

Games, robotics, and interactive demos / Changing candidates

Use: Tetris controller judges changing placements with several choice questions.

Already visible: Per-call JSONL and game state.

Observer could help: Inspect each question distribution alongside the blended final decision.

Limit: Selected action is computed from multiple distributions, not necessarily an individual argmax.

[Implementation evidence](#) · [Project README](#)

082 standardagents/jevpilot

Games, robotics, and interactive demos / Changing candidates

Use: Driving simulation chooses legal routes and speeds when alternatives exist.

Already visible: Live payload display and server cost tracking.

Observer could help: Compare decision phases and candidate-aware uncertainty over trajectories.

Limit: Single-candidate steps skip Jev and are invisible to a proxy.

[Implementation evidence](#) · [Project README](#)

083 gaborishka/jevtown

Games, robotics, and interactive demos / Configured rules

Use: Audience simulator defines reactions using configurable persona and content presets.

Already visible: Reaction summaries and interactive audience views.

Observer could help: Compare preset versions and recurring response distributions by cohort.

Limit: Simulated reactions are model outputs, not measured audience outcomes.

[Implementation evidence](#) · [Project README](#)

084 spoonnotfound/soupbase

Games, robotics, and interactive demos / Fixed definitions

Use: Puzzle host answers and grades guesses using bounded question definitions.

Already visible: Interactive answers and confidence feedback.

Observer could help: Track host and guess-grading families across puzzle sessions.

Limit: Puzzle context affects meaning; native Vercel evaluation and remote deployment need adapters.

[Implementation evidence](#) · [Project README](#)

Appendix / Projects 085-088

Static review · Proposed benefits are hypotheses · Code and README links are pinned to reviewed commits.

085 achimala/jev-paint

Media and creative tools / Indexed/contextual instances

Use: Paint experiment issues per-pixel or channel decisions with embedded coordinates.

Already visible: Rendered image output and interactive controls.

Observer could help: Group pixel templates into channel or representation statistics.

Limit: High fan-out stresses storage; coordinates cannot each create an independent dashboard.

[Implementation evidence](#) · [Project README](#)

086 valentynkit/jev-skip

Media and creative tools / Indexed/contextual instances

Use: Video extension classifies indexed segments for skipping.

Already visible: Heatmap, thresholds and labeled reliability examples.

Observer could help: Compare segment-rule versions and reviewed false skips.

Limit: Keep video context and segment indices; extension transport needs testing.

[Implementation evidence](#) · [Project README](#)

087 cocktailpeanut/jevthoven

Media and creative tools / Changing candidates

Use: Music generator selects bounded plan, instrument, bar and note choices.

Already visible: Persisted decision receipts and composition UI.

Observer could help: Compare distributions and request costs by musical decision kind.

Limit: Candidate meanings vary by composition stage; output quality needs listener labels.

[Implementation evidence](#) · [Project README](#)

088 Dearest/plotveil

Media and creative tools / Indexed/contextual instances

Use: Spoiler filter evaluates comments with video context and comment references.

Already visible: Hidden-until-judged interface and evaluation script.

Observer could help: Group spoiler-check templates and attach user corrections.

Limit: Remote worker needs export; video/title context cannot be erased without a scoped adapter.

[Implementation evidence](#) · [Project README](#)

Appendix / Projects 089-092

Static review · Proposed benefits are hypotheses · Code and README links are pinned to reviewed commits.

089 harshil1712/slidepilot

Media and creative tools / Fixed definitions

Use: Speech-driven slides judge completion and the next presentation action.

Already visible: Presentation controls and threshold policy.

Observer could help: Compare threshold versions against actual early or late advances.

Limit: Hosted worker needs import; execution timing is application data.

[Implementation evidence](#) · [Project README](#)

090 kitze/unclutter

Media and creative tools / Indexed/contextual instances

Use: Browser extension classifies page elements before creating hiding rules.

Already visible: Persisted reusable rules and browser results.

Observer could help: Compare per-site classification families and reviewed hiding mistakes.

Limit: Rule reuse bypasses Jev; element IDs and page context need an adapter.

[Implementation evidence](#) · [Project README](#)

091 FirasSX914/Janus

Evaluation and calibration / Study/benchmark variants

Use: Studies routing between Jev decisions and teacher fallbacks.

Already visible: Labeled evaluations, threshold sweeps and reports.

Observer could help: Import runs for comparable question histories and disagreement inspection.

Limit: Large overlap with existing calibration work; teacher agreement is not automatically correctness.

[Implementation evidence](#) · [Project README](#)

092 chunxiaox/nautilus-compass

Evaluation and calibration / Study/benchmark variants

Use: Contains a hosted Jev calibration study with controlled synthetic questions.

Already visible: Study report and raw run artifacts.

Observer could help: Compare experiment cells and wording variants in a local viewer.

Limit: Scope is this study; the main memory project was not established as a Jev runtime integration.

[Implementation evidence](#) · [Project README](#)

Appendix / Projects 093-096

Static review · Proposed benefits are hypotheses · Code and README links are pinned to reviewed commits.

093 jev-ids/jev-ids

Evaluation and calibration / Fixed definitions

Use: Intrusion-detection experiments ask attack and category questions against labeled data.

Already visible: Prediction JSONL, labels and dataset metrics.

Observer could help: Compare predicate and taxonomy versions with measured false positives.

Limit: Definitions refer to task instructions in state; definition-only matching can conflate tasks.

[Implementation evidence](#) · [Project README](#)

094 RINNECODER/jev-behavior-study

Evaluation and calibration / Study/benchmark variants

Use: Measures sensitivity to framing and choice presentation.

Already visible: Raw responses and experiment reports.

Observer could help: Inspect paired variants while preserving original question and option ordering.

Limit: Never normalize away the experimental variable; study findings were not independently reproduced.

[Implementation evidence](#) · [Project README](#)

095 AbdelStark/jev-benchmarks

Evaluation and calibration / Study/benchmark variants

Use: Benchmark adapter creates choices from each dataset example's labels.

Already visible: Raw probabilities, usage and calibration reports.

Observer could help: Import standardized run manifests for cross-run inspection.

Limit: Per-example candidate domains vary; avoid building a duplicate benchmark engine.

[Implementation evidence](#) · [Project README](#)

096 baibizhe/jev-decision-benchmarks

Evaluation and calibration / Study/benchmark variants

Use: Report repository rebuilds and checks saved Jev benchmark evidence.

Already visible: Saved tables, case evidence and offline report builder.

Observer could help: Import saved cases to inspect disagreement and question definitions.

Limit: Artifact study, not a verified live runtime integration; report code was read, not executed.

[Implementation evidence](#) · [Project README](#)

Appendix / Projects 097-100

Static review · Proposed benefits are hypotheses · Code and README links are pinned to reviewed commits.

097 vinilana/jev-eval-agent

Evaluation and calibration / Fixed definitions

Use: Evaluation agent selects from mocked tools and checks completion.

Already visible: Trace JSONL and evaluation dashboard.

Observer could help: Separate tool-selection uncertainty from task-completion outcomes.

Limit: Mocked-tool evaluations do not establish production tool reliability.

[Implementation evidence](#) · [Project README](#)

098 yodablocks/jev-orderby-bench

Evaluation and calibration / Study/benchmark variants

Use: Studies ranking and calibration using typed questions and wording variants.

Already visible: SQLite cache, raw reports and metrics.

Observer could help: Inspect run manifests, wording versions and retained distributions.

Limit: Already has extensive evaluation; cache and experiment order must be preserved.

[Implementation evidence](#) · [Project README](#)

099 zhuyansen/jev-search-rerank-eval

Evaluation and calibration / Indexed/contextual instances

Use: Search benchmark scores indexed candidates and also supports Jev-generated labels.

Already visible: Ranking evaluations, label records and provider cost fields.

Observer could help: Separate ranker and judge families and compare rubric versions.

Limit: Keep label provenance to avoid presenting self-agreement as independent accuracy.

[Implementation evidence](#) · [Project README](#)

100 abhixhek/jevcal

Evaluation and calibration / Study/benchmark variants

Use: Calibrates thresholds and checks drift with provider and teacher labels.

Already visible: CLI reports and CI-oriented metrics.

Observer could help: Import captured decisions for local version inspection and exports.

Limit: Complement its calibration tooling; teacher labels require explicit provenance.

[Implementation evidence](#) · [Project README](#)